

# 3PaaS: Privacy-Preserving Post-Compromise Security as a Service

Cas Cremers<sup>1</sup>, Abhinav Nakarmi<sup>2</sup>, Aleksi Peltonen<sup>3</sup>, and Eyal Ronen<sup>4</sup>

<sup>1</sup>CISPA Helmholtz Center for Information Security, [cremers@cispa.de](mailto:cremers@cispa.de)

<sup>2</sup>University of Michigan, [nakarmi@umich.edu](mailto:nakarmi@umich.edu)

<sup>3</sup>University of Aberdeen, [aleksi.peltonen@abdn.ac.uk](mailto:aleksi.peltonen@abdn.ac.uk)

<sup>4</sup>Tel Aviv University, [eyalronen@tauex.tau.ac.il](mailto:eyalronen@tauex.tau.ac.il)

Version 1.0  
July 1, 2026

## Abstract

Post-Compromise Security [20] (PCS) expresses that even after a party has been compromised, it may be able to recover (or “heal”) security afterwards. PCS has been extensively studied, and many end-to-end secure messaging applications include mechanisms, such as the double ratchet, to achieve it.

However, current approaches only work partially, and their healing effects are extremely limited. For example, recent work [27] showed that due to real-world constraints such as resilience against state loss, even highly secure messengers such as Signal may not achieve it in practice. Moreover, since healing is session-specific, healing effects do not carry over to newly spawned sessions, different groups [24], or different services that use the same identity.

In this work, we tackle these issues by designing the first protocol that can provide PCS as a Service for identities through a third party. The major challenge is privacy: achieving PCS requires regular updates among participants, and involving a third party can lead to significant privacy concerns. Moreover, the type of update that PCS requires (updating a user’s secret, but *only once*) seems to require servers to verify the users’ identities in a way that contradicts the unlinkability required for privacy: the server should not learn anything about the users’ activities.

We develop the 3PaaS protocol, including the first efficient zero-knowledge proofs for blind signatures, to achieve our goals, and even allow for revocation, without revealing the identity to the server. We formally analyze our protocol for high assurance, provide an implementation of our novel ZK building blocks, and show how our protocol could be used with a messaging application.

## 1 Introduction

Modern cryptographic protocols often require strong security and privacy guarantees, especially in the context of secure communication over end-to-end encrypted (E2EE) messaging services. One of the core requirements is *post-compromise security* (PCS) [20], which refers to the ability to recover, or *heal*, from protocol participants getting compromised. Specifically, the goal is that even when the long-term keys of a party have been revealed, it should be able to derive and distribute new keys in a way that deprecates old ones, thus preventing an adversary from hijacking sessions in the future.

The notion of PCS has been extensively studied in the literature and many end-to-end encrypted messaging services implement mechanisms, such as the double ratchet [50], to achieve it. However, in practice, proper PCS is difficult to achieve and often works only partially with limited healing effects. Notably, a system might not achieve PCS, even when the underlying cryptographic protocol guarantees it. For example, recent work [26, 27] studying Signal concluded that due to real-world constraints, such as resilience against state loss, it was not able to correctly achieve PCS in practice. Moreover, since the recovery mechanism is session-specific, its healing effects do not carry over to newly spawned sessions, different groups [24], or other services that use the same identity.

Another challenge related to secure communication, is that distributing and verifying long-term keys is difficult. This is because, for every new session that requires authentication, all parties must exchange public keys and confirm each other’s identities. Current solutions based on out-of-band authentication (e.g., scannable QR codes) or publicly auditable key transparency systems [14, 52, 49, 44, 43] are either

difficult to use or require users to trust the service provider. Furthermore, since they are typically implemented and maintained separately for each application, interoperability between different services is not achievable without modifications.

The Digital Markets Act (DMA) [33] requires end-to-end encrypted messaging apps defined as “gate-keepers” (e.g., WhatsApp), to support interoperability with other applications such as Signal. In [45], the authors note that “...this interoperability mandate represents a fundamental change to the basic centralized architecture of today’s E2EE systems.”

In this paper, we address these challenges by designing the first protocol that provides *post-compromise security as a service* through an independent third party. Beyond solving the aforementioned limitations, we also tackle previously unsolved privacy issues. Despite the regular updates required to achieve PCS, our solution provides unlinkability and strong privacy guarantees that prevent even a compromised service provider from revealing confidential information about its users. Furthermore, since our solution is independent of the application it is used for, it can support interoperability between services.

One major challenge for our solution is to ensure a credential is only given to an authorized party with a unique identity, without revealing any of the identities to the server, while maintaining PCS. E.g., we want the server to provide a credential that attests the user’s phone number, without the server learning the phone number or even being able to link two requests from the same user, while still allowing the user to heal after a compromise. A similar challenge was tackled by the TokenWeaver [25] protocol that provides PCS while preserving “group unlinkability” [8]. However, the TokenWeaver protocol can achieve this notation of group unlinkability exactly because its privacy-preserving credential only needs to attest that the user is a member of some *group* (i.e., the set of valid FIDO2 hardware tokens). For our objective, we need to attest to the user’s unique identifier without revealing the identity. To overcome this challenge, we design the first efficient zero-knowledge proofs for blind signatures based on RFC 9474 [31].

Conceptually, our solution leverages a trusted service to provide authenticity with PCS. This authenticity with PCS can then be combined with standard mechanisms for Perfect Forward Secrecy (PFS) (such as X3DH) to also obtain PCS for confidentiality.

To confirm that our solution provides the claimed security and privacy guarantees, we formally model and analyze it with two state-of-the-art protocol verification tools, the TAMARIN prover [56] and DeepSec [18, 17]. Furthermore, we also show that it can easily be integrated into secure messaging applications, like Signal, and that the computational overhead of the cryptographic proofs and derivations are negligible.

**Contributions** Our main contributions are the following:

1. We propose 3PaaS – the first privacy-preserving protocol for providing post-compromise security as a service. As a novel result, we show that using zero-knowledge proofs and blind signatures, we are able to offer strong privacy guarantees against a potentially compromised service provider. Furthermore, our solution offers cross-application interoperability by separating the identity attestation provider from the rest of the application infrastructure, and supports multiple independent attestation servers.
2. We design the first efficient zero-knowledge proofs for blind signatures based on RFC 9474 [31] and advance the optimization of non-membership proofs, which are of independent interest for a large range of applications.
3. We implement and benchmark the cryptographic primitives used in the proposed protocol to demonstrate that it meets the expected efficiency requirements. We argue that the overhead caused by providing privacy and unlinkability is negligible, even in the case of secure messaging. We provide the benchmarking code at [28].
4. We formally model the proposed protocol and prove its core security and privacy properties using two state-of-the-art protocol verification tools, the TAMARIN prover and DeepSec. In addition to confirming the claimed properties of 3PaaS in isolation, our models can also be integrated into larger case studies to analyze its impact on specific applications. We provide our models and detailed instructions at [29].
5. We outline how our solution can be adapted to modern secure messaging applications, like Signal, with minimal changes, while maintaining backward compatibility.

**Outline** We provide background and summarize related work in Section 2. In Section 3, we explain the main security and privacy requirements. In Section 4, we introduce our solution and informally explain

how it meets the expected requirements. In Section 5, we describe our proof-of-concept implementation and present benchmarks for the zero-knowledge proofs. In Section 6, we formally analyze the proposed protocol and verify its core security and privacy properties. In Section 7, we discuss practical applications of our protocol and future work, and conclude in Section 8.

## 2 Background

In this section, we provide background on topics related to our proposed solution, including PCS, cryptographic primitives, and the TokenWeaver [25] protocol. We also summarize related work.

### 2.1 Post-Compromise Security

Post-Compromise Security (PCS) [20] describes the security guarantees achieved by a protocol after the secrets of a participant have been revealed. Notably, this can include long-term keys, which is often ignored in the design and proofs of PCS mechanisms. The goal is to allow compromised parties to recover by updating their keys in a way that prevents an outsider from hijacking their future communication sessions. This is typically achieved through a *healing* step that updates or verifies new credentials and invalidates old ones.

In practice, the type of PCS we can achieve depends on how frequently the healing step is performed. For this work, we consider two variations:

1. **Coarse-grained PCS** – The healing step is performed at pre-determined time intervals (e.g., once an hour, a day, or a week). For example, this can be instantiated by periodically rotating keys (e.g., the parties public keys). If PCS is provided through a third party (e.g., a third party that acts as a PKI), it learns only coarse-grained availability information about the client, including the times it was online to perform the healing process. However, healing is only possible at the end of each interval.
2. **Fine-grained PCS** – The healing step is performed each time a client initiates a new session (e.g., a new ratchet chain). For example, this can be instantiated by requesting the third party to provision new credentials for each new session. This enables faster healing and provides the best PCS guarantees. However, in the case of a third-party service provider, it requires additional measures to prevent the leakage of sensitive metadata.

### 2.2 Cryptographic primitives

The privacy guarantees provided by our protocol are achieved with the use of zero-knowledge proofs and blind signatures. In this section, we briefly summarize the core properties of these primitives. For a more detailed explanation of how we implement and optimize them, see Section 5.

**Zero-Knowledge Proofs** A zero-knowledge proof (ZKP) allows a prover to convince a verifier the correctness of a statement, without revealing anything other than its validity. More precisely, suppose a prover wants to show that it knows a public input and a private witness  $((x, y), w)$ , s.t.,  $y \leftarrow f(x, w)$  on a public function  $f$  (also referred to as a “statement”). The prover computes a proof  $\pi$  that convinces the verifier that it knows  $w$  without revealing it. The property of the proof system that guarantees that the prover knows a witness  $w$  is called *knowledge soundness*, and the property that hides the witness is called *zero-knowledge*.

A non-interactive ZKP that produces a succinct proof is known as a *Zero-Knowledge Succinct Non-Interactive Argument of Knowledge* (zkSNARK). Most modern zkSNARKs require the public function  $f(\cdot)$  to be compiled into a generalization of an arithmetic circuit, called R1CS. For simplicity, we refer to them as *circuits* in this paper. In our protocol, we use Mirage [41], since it supports verifier randomness, which is required for efficient arithmetic operations that we use [59]. Mirage has a trusted setup, which processes  $f(\cdot)$  to produce a prover key and a verifier key. We make the service provider run the trusted setup since it is the party demanding proofs from the clients. The service provider can then transfer the prover key to the client during the sign-up phase.

**Blind signatures** A blind signature scheme [15] allows signing a message without learning its content, such that the signature can later be *unblinded* and verified like a regular digital signature. It consists of four algorithms: (**sign**, **verify**, **blind**, **unblind**). The first two represent a classic signature scheme.

The third one,  $\text{blind}(m, sk_b)$ , implements the blinding functionality by using a *blinding key* ( $sk_b$ ) that conceals the message ( $m$ ). Once the blinded message has been signed, it can be unblinded with the same key as follows:  $\text{unblind}(\text{sign}(\text{blind}(m, sk_b), sk_s), sk_b) = \text{sign}(m, sk_s)$ , where  $sk_s$  is the long-term key of the signer. RFC 9474 [31] is a standard for the secure implementation of blind signatures.

## 2.3 TokenWeaver

Our solution is inspired by the TokenWeaver [25] protocol. TokenWeaver is a privacy-preserving remote attestation method that provides post-compromise security for trusted execution environments (TEEs). The protocol is built on two core mechanisms: a linkable authorization chain used by the service provider to keep track of valid TEEs, and an unlinkable chain that provides PCS, privacy, and unlinkability. The linkable chain enables efficient compromise detection and revocation but does not hide the identity of the requesting TEE from the service provider. However, each node in it can be used to instantiate a privacy-preserving unlinkable chain that provides fine-grained PCS by providing certificates to attestation keys.

In order to hide the attestation keys and the identity of the requesting TEE from the service provider, TokenWeaver relies on blind signatures. After generating a new key pair, the TEE establishes a secure connection to the service provider and authorizes itself with a single-use token. It then sends blinded versions of its new public key and a fresh authorization token, which the service provider signs. After receiving the (blinded) signatures, the TEE unblinds them, giving it (1) an attestation key pair and a valid certificate of the public key, and (2) an unspent authorization token and a signature of it. Since the service provider does not learn the token until the next round, it is unable to link multiple protocol executions or key pairs to a single TEE.

Our solution reuses the authorization tokens and blind signatures from TokenWeaver. However, TokenWeaver aims to solve a different problem than ours, and thus its security goals are simpler to satisfy. In the TokenWeaver protocol, the service provider requires the client to only prove it is *some* valid TEE, which has a public serial number on the linkable chain. However, we further require that each client can only request signatures for its own public identifier. In the linkable version of 3PaaS, we can simply send the client identifier to the service provider to verify. In the unlinkable version, this is, however, not possible. Instead, we rely on zero-knowledge proofs that allow the client to prove to the service provider that its real identity is being signed, without revealing it.

## 2.4 Related work

**Post-Compromise Security** The concept of PCS has been extensively studied and implemented, especially in the context of end-to-end encrypted messaging [50, 5, 21, 38, 2, 10, 19, 26, 7, 57, 46]. However, current approaches are typically application-specific and work only partially. In particular, their healing effects are limited: Recent work [27] concludes that due to real-world constraints, such as resilience against state loss, even highly secure messengers like Signal may not achieve PCS in practice. Moreover, since the mechanism is session-specific, any healing effects do not carry over to newly spawned sessions, different groups [24], or other services that use the same identity.

**Zero-Knowledge Proofs** PKCS1v1.5 and EMSA-PSS are two widely used message encoding schemes for RSA signatures. While PKCS1v1.5 has been studied and implemented in the context of zero-knowledge proofs [59, 60], the same is not true for PSS encoding. Recent work [59] suggests that a naive implementation of PSS encoding is expensive. Recent RFCs and applications, such as the blind-signature RFC 9474 [31], now use PSS encoding instead of PKCS1v1.5 as it yields a scheme with a tight reduction to the hardness of inverting RSA [59].

**Key Transparency** In the context of secure, end-to-end-encrypted messaging, identity attestation and key distribution is commonly solved with *Key Transparency* (KT) [16, 14, 52, 49, 44, 35, 43, 51]. In a KT system, the service provider maintains a publicly auditable directory of user identifiers mapped to their current public keys. The system is designed such that it can only be appended, and each identity can have at most one valid public key at any given time. However, although KT allows users to verify remote parties' public keys and identities, it does not provide PCS.

### 3 Security and Privacy Goals

Informally, our goal is to design a protocol that allows a trusted third party to provide PCS by offering short-term credentials for identifiers even in case of long-term key compromise and active attackers. For example, in the context of secure messaging, when initiating a new session with a remote party, the sender’s long-term private key is used to authenticate the initial message (e.g., implicit authentication via the key derivation in Signal’s X3DH protocol [50]). If a party’s long-term key is compromised, an active adversary can impersonate this party and open new sessions. Although X3DH provides PCS for the encryption keys during the session, it does not provide any guarantees to new sessions in case of compromise of long-term keys, since adversaries can just create pre-keys and sessions on behalf of the compromised party [26].

Our solution can prevent such attacks, by requiring the receiver to only accept the message if the new session request includes a valid attestation credential from the server. This credential will be either very short-lived, or even bounded to the specific new session request. The PCS will be provided by the protocol that allows the party to request a new credential. Besides requiring the protocol to securely authenticate such requests and provide PCS, we further expect it to do so without leaking anything about the values it signs or the identity of the requesting party even to a malicious server.

Our protocol can support multiple independent servers. The receiver could determine which attestation services it trusts, and even require certificates from multiple independent servers. As our protocol is privacy preserving, adding more services only adds to the security of the system, without any loss of privacy.

More formally, the problem we are solving is as follows: *Given a protocol  $P2$  that involves identity authentication, design protocols  $P1$  and  $P2'$  such that 1.  $P2'$  is a modification of  $P2$  with minimal changes, and 2.  $P1$  allows a trusted third party to provide a credential that can be used in  $P2'$  to provide PCS to the party’s identity.*

The main requirement for  $P1$  is to enable PCS for  $P2'$ , such that after a party  $A$  heals, only  $A$  is able to authenticate with  $P2'$  in the future, even if its long-term keys have previously been leaked. Any clone or adversary that could previously impersonate  $A$  is now unable to do so, since opening a new session will require a new credential, which after each healing step can only be requested by one successor. Similarly to TokenWeaver, we also provide the two following properties:

1. **Clone detection** – If an adversary clones a party and request a new token before the honest party they will “lock out” the honest party. This will be detected by the honest party the next time they will try to run the protocol.
2. **Active revocation** – After a clone has been detected, the honest user can use an out-of-band channel to ask the server to revoke the compromised identifier. After this revocation, the adversary will not be able to request new credentials.

Additionally, the system should offer some level of privacy, depending on the use case. We consider the following two properties:

1. **Privacy** – The  $P1$  protocol does not leak any information to the PCS server about  $P2'$ , including the identities of the participating clients.
2. **Unlinkability** – The PCS server cannot link two executions of  $P1$ . Even if a malicious PCS server colludes with malicious  $P2'$  participants, it cannot link any two executions of  $P1$  unrelated to the malicious participants. <sup>1</sup>

**Coarse- vs. fine-grained PCS** While providing coarse-grained PCS is relatively straightforward, achieving our goal of privacy-preserving fine-grained PCS is a significantly more challenging task. We can provide coarse-grain PCS by periodically rotating the long-term public keys. This can be done, for example, by using the linkable chain solution from TokenWeaver to allow the user to request a credential signing their new public key while providing PCS. As we will see, our full 3PaaS solution that provides fine-grained PCS in a privacy-preserving manner requires more complicated tools such as zero-knowledge proofs.

---

<sup>1</sup>I.e., if the credentials provisioned by the two executions of  $P1$  were not used in an execution of  $P2'$  that involves malicious participants, they will remain unlinkable.

### 3.1 Threat model

In our security analysis, we use the standard Dolev-Yao [32] model of an adversary that controls the network, is able to intercept and inject messages but not break cryptographic primitives. Furthermore, we also consider the cases where the server (representing the PCS provider) and/or one of the communicating parties is compromised. We note that, in the case of a malicious server, we lose PCS, but not privacy. Notably, we show that even a malicious server unable to weaken the privacy guarantees provided by our solution for connections used to provision credentials for sessions between honest clients.

We assume unilaterally authenticated secure channels between the clients and the server (e.g., using TLS), where the server is authenticated. We also assume that these authenticated channels are unlinkable and do not leak any information about the clients' identities. This could be achieved using the Tor network, or some other relaying party that is only trusted to hide the communication metadata (e.g., the Signal server).

## 4 Post-Compromise Security as a Service

We now introduce 3PaaS – *Privacy-Preserving PCS as a Service* protocol. 3PaaS is inspired by the PCS token chain idea from TokenWeaver [25], where each party has a single-use token that can be spent to request a credential from the server together with a new token to be spent in the next request. In our protocol, clients request credentials from the server to ensure PCS, e.g., when initializing a new session with a remote party. While TokenWeaver only allows the party to prove that it belongs to some authorized group, the credential provisioned by the 3PaaS server binds the new session to some *specific* known long-term identifiers, such as identity public keys (currently used in Signal and MLS) or UUIDs (e.g., a phone number or an email address), such that it is only valid for the relevant parties. Allowing this identity “binding” while providing privacy to the users require us to use more advanced techniques such as zero-knowledge proofs.

We assume that clients have an out-of-band channel for authentication with the server that can be used to register new clients, and to ask for revocation in case of client compromise. We exclude from our design the specific details of this out-of-band channel, but note that such channels are already in use today by most large service providers to allow honest users to regain access to their account after a compromise. After the initial registration, our protocol allows clients to request new credentials in a *privacy-preserving* manner while maintaining PCS.

### 4.1 Server requirements

The high-level goal of the protocol is for a party  $A$  to be able to request a credential from a server, such that it authorizes them to open a new session with some remote party  $B$ . The server needs to verify that only  $A$  or  $B$  can request a signature for a session between them, i.e., an outsider should not be able to receive a credential that does not involve itself. Each request by  $A$  should provide PCS healing, such that after receiving a credential, any other clone would not be able to ask for more credentials on behalf of  $A$ . For this the server should run the following checks:

1. **Double request prevention** – Each request for a new credential should have exactly one predecessor, i.e., the chain of requests should not split.
2. **Identifier consistency** – Each request chain should be “bound” to a single party identifier, such that all request in the chain of requests should have come from the same party.
3. **Session binding** – A credential that allows opening a new session between  $A$  and  $B$  should be bound to those two parties (such it cannot be used to open a new session with other parties), and at least one of the identifiers (either  $A$  or  $B$ ) should be the same as the one that the request chain is “bound” to.<sup>2</sup>
4. **Revocation** – A request should be rejected if the party's identifier has been revoked.

We describe two variations of 3PaaS. First, we outline a simple linkable variant of the protocol and then continue to present the full unlinkable solution.

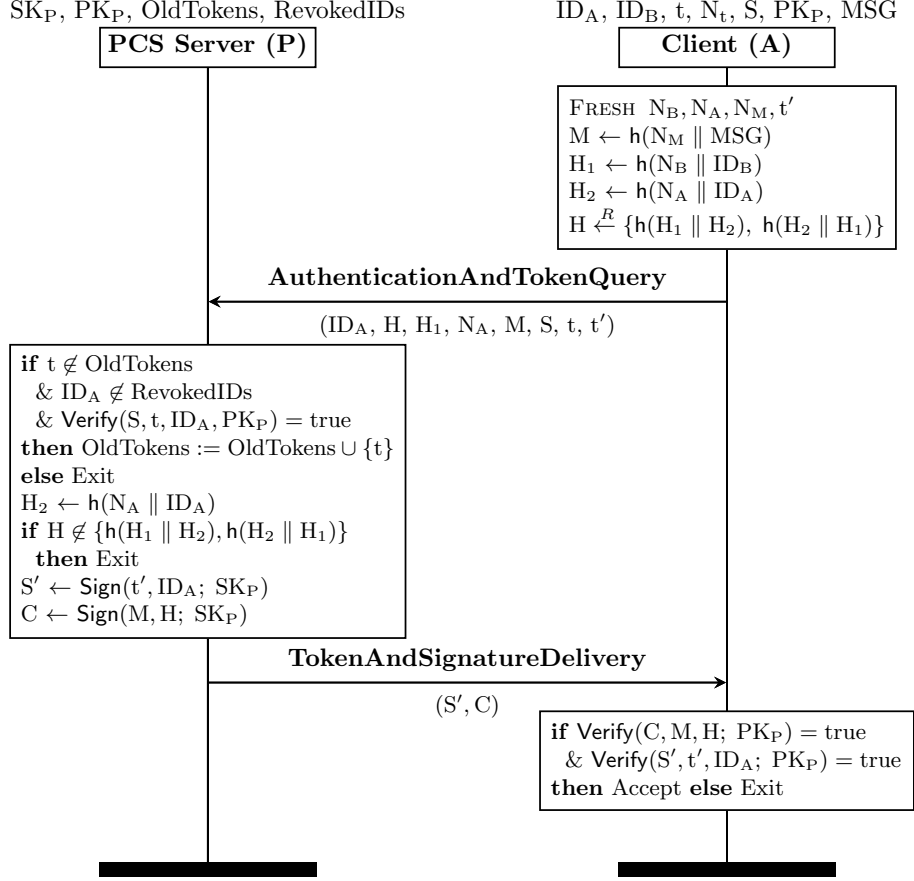


Figure 1: Linkable version of 3PaaS showing the PCS step for the initiator. MSG is the first message that Client A intends to send to Client B.

## 4.2 Linkable version

In the linkable version of 3PaaS, the PCS server learns the identity of the client requesting a credential but not the contents of the message it signs or the identity of the message receiver (i.e., the remote party). Figure 1 shows the PCS step of the protocol.

We assume that the client has a valid one-time token  $t$  and a signature  $S$  previously signed by the server that validates the token and binds it to the client's identifier  $ID_A$ . The first token is provisioned by the server during the registration process. When requesting a new credential, the client sends its current token ( $t$ ) and signature ( $S$ ), a new token ( $t'$ ), its own identifier ( $ID_A$ ), and the value to be signed ( $M$ ). The value  $M$  should include a hash of the first message that the client intends to send. The credential will include a signature on  $M$  together with the identifiers of both the sender and the receiver. This binds the credential both to the specific new session request and the parties identifiers. If the current token has not been spent yet, it is bound to the identifier of the requesting client, the server responds with a new credential  $C$  that signs message  $M$ , and a new signature  $S'$  that validates the new token  $t'$ .

### 4.2.1 Privacy

To hide the identifier of the remote party, the client only sends a hash of a random nonce  $N_B$  concatenated to the remote party identifier  $H_1 = h(N_B || ID_B)$ . This allows the credential to be bound to the identity of the remote party without revealing it to the server. The order of the hashes is picked at random, such that any credential that allows opening a new session between party  $A$  and  $B$  could have been requested by either party. This means that by itself, credential does not reveal if it was requested by party  $A$  or  $B$ . We provide this property as it can be used to allow for some forms of deniability depending on the properties of P2'. Exact formalization and analysis of such deniability is outside the scope of this work.

<sup>2</sup>This means that by itself, credential does not reveal if the session was initiated by  $A$  or  $B$ . See Section 4.2.1.

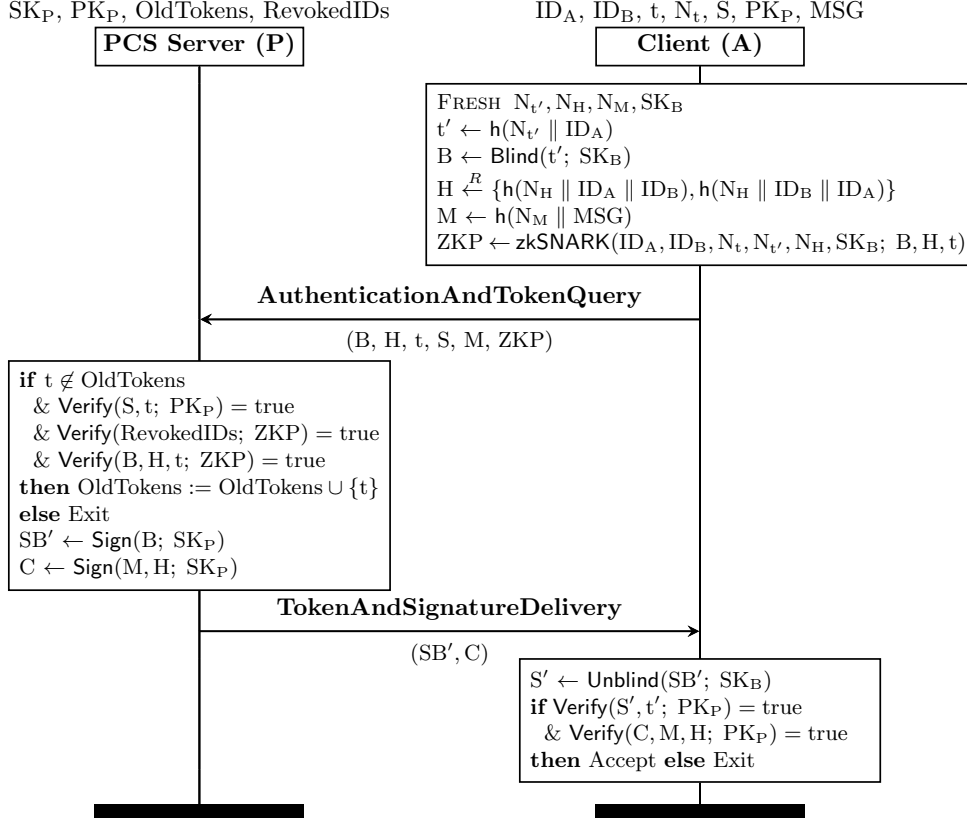


Figure 2: Unlinkable version of 3PaaS showing the PCS step for the initiator. MSG is the first message that Client A intends to send to Client B.

### 4.3 Unlinkable version

TokenWeaver [25] achieves unlinkability together with PCS using so called “unlinkable” chains based on authorization tokens signed using blind signatures. The full unlinkable version of 3PaaS described in Figure 2 uses a similar approach together with ZK proofs to bind all requests and credentials to the client’s unique identifier.

**Client value generation** For a client  $A$ , both the old token  $t$  and the new token  $t'$  are generated as hashes of its identifier  $ID_A$  and a random nonce ( $N_t$  or  $N_{t'}$ ):

$$t \leftarrow h(N_t \parallel ID_A); \quad t' \leftarrow h(N_{t'} \parallel ID_A) \quad (1)$$

The freshly generated nonces are used to hide the value of the identifier. In order to maintain unlinkability, we need to get the server signature on the new token  $t'$  without revealing its value. For this we use blind signatures, where the client generates an ephemeral blinding key  $SK_B$  and uses it to “blind” the new token  $t'$  before sending it to the server for signing:

$$B \leftarrow Blind(t'; SK_B) \quad (2)$$

Furthermore, we calculate the hash of both parties’ identifiers. As in the linkable version, we choose a random ordering of both parties identifiers:

$$H \xleftarrow{R} \{h(N_H \parallel ID_A \parallel ID_B), h(N_H \parallel ID_B \parallel ID_A)\} \quad (3)$$

where  $N_H$  is a freshly generated nonce and  $ID_B$  is the public identifier of the remote party.

**Zero-knowledge proofs** To prove that the same (unrevoked) identifier was used in all calculations, the client provides a zero-knowledge proof of the following:

1. The same identifier  $ID_A$  was used to construct  $t, t'$  and  $H$ .

2.  $B$  is a blinding of  $t'$  that was constructed from  $ID_A$  (as only the blinded value  $B$  is sent to server, instead of  $t'$ ).
3. The identifier  $ID_A$  is not in the public Merkle tree of revoked identifiers.

Full details of the zero-knowledge proof are provided in Section 5. Note that we indeed fulfill all the requirements described in Section 4.1. We prevent double request by using one-time tokens and the Zero-knowledge proof verifies identifier consistency, session binding, and checks for revocation.

**Credential and token provisioning** Upon receiving the client's request, the server checks that the old token  $t$  was never used before, that its signature is valid, and that the zero-knowledge proof verifies. If the request passes all of the checks, the server will send the client its signature on the blinded value  $B$  and a new credential that signs the two hash values  $H$  and  $M$ . Finally, the client unblinds the server signature to get the signature  $S'$  on the new token  $t'$  and validates both  $S'$  and the credential. Note that without this verification step on the blind signature, we may lose the unlinkability property, as a malicious server may link sessions using malformed signatures.

**Unlinkable revocation** In order to prevent malicious parties from requesting signatures with compromised tokens, we implement a revocation mechanism that allows the user to deprecate identifiers. The revocation procedure is done over an out-of-band channel, which the service provider can implement as they choose. Once the server receives a (valid) revocation request, it adds the identifier to a public Merkle tree. In each signature request, clients are required to provide a zero-knowledge proof showing that the identifier they are requesting a signature for, is not part of the tree. We describe this mechanism in Section 5.

## 5 ZK Proof Instantiation

In this section, we describe the zero-knowledge proof statements required to implement the unlinkable version of 3PaaS. We also explain how we implement and optimize them, and provide benchmark results. The code and results are available for review at [28].

### 5.1 Proof statement

Recall that we use ZK proofs to prove statements regarding the client's ID, without revealing it to the server. When requesting a new credential, we want to prove that the old token, the new token, and the new session we intend to open, are all "bound" to the same ID, which we denote as  $ID_A$ . Moreover, we want to prove that this ID was not revoked by the server.

First, the client proves that it knows a nonce  $N_t$ , which together with the identifier  $ID_A$  was used to compute the old token  $t$ . Second, the client proves that the same identifier  $ID_A$  was used as one of the two identifiers in the derivation of the public witness  $H$  (without revealing which of them). Third, the client proves that it knows the nonce  $N_{t'}$  and the blinding key  $SK_B$  that together with identifier  $ID_A$  were used to compute the new token  $t'$ , and that it correctly blinded  $t'$  to produce the public value  $B$ . Finally, the client proves  $ID_A$  is not revoked.

The client proves the following statements using zkSNARKs with the private witnesses ( $N_t$ ,  $N_{t'}$ ,  $ID_A$ ,  $ID_B$ ,  $N_H$ ,  $SK_B$  - all marked in teal) and the public witnesses ( $t$ ,  $B$ ,  $H$ ,  $RT$ ):

1.  $t \leftarrow h(N_t \parallel ID_A)$ ;
2.  $H \in \{h(N_H \parallel ID_A \parallel ID_B), h(N_H \parallel ID_B \parallel ID_A)\}$ ; and
3.  $B \leftarrow \text{Blind}(t'; SK_B)$ , where  $t' \leftarrow h(N_{t'} \parallel ID_A)$ .
4. Finally, the client also proves that the identifier  $ID_A$  has not been revoked by the service provider, i.e., it is not in the revocation tree  $RT$  published by the server.

Assuming a collision-resistant hash function, proving that the tokens were derived correctly from the same private  $ID_A$  proves that they are all bounded to the same ID. While the ZK proofs on hash functions required for steps 1 and 2 are relatively straightforward, efficient ZK proofs for blind signatures and non-membership are slightly more complex, and require some dedicated design and optimization to allow for practical proof generation time.

---

**Algorithm 1** The Blind/Unblind functionality.

---

**function** Blind( $mHash, (e, N), salt$ ):

$em \leftarrow \text{Encode}(mHash, salt)$

$r \xleftarrow{R} Z_N^*$

$SK_B \leftarrow r^e \bmod N$

$B \leftarrow em \cdot SK_B \bmod N$

**return**  $B, r$

**function** Unblind( $BlindSig, r, N$ ):

$Sig \leftarrow BlindSig \cdot r^{-1} \bmod N$

**return**  $Sig$

---

## 5.2 ZK proofs for RSA blind signatures

To maintain unlinkability, we use blind signatures to request a signature on the new token  $t'$ , without revealing it to the server.

We use RSA blind signatures described in RFC 9474 [31] and the PSS encoding algorithm [53], which describe the computation of the blinded-message  $B$  from a hash on a message  $mHash$  (which in our case is the token  $t' = h(N_{t'} \parallel ID_A)$ )<sup>3</sup>. The keys used for the blind signature are the standard RSA private public key pair, i.e., a public modulus  $N = p \cdot q$ , such that  $p$  and  $q$  are primes, a public exponent  $e$ , and a private key  $d$  such that  $d = e^{-1} \bmod \phi(N)$ . For our implementation, we choose a 2048 bit modulus.

While more ZK-proofs-friendly blind signature schemes may be used, we prefer to use the standardized scheme and believe that the design of ZK proofs for RSA-PSS is a useful independent contribution. Moreover, due to the modular approach we use, 3PaaS's design and proofs can be easily adapted to use any blind signature scheme.

Algorithm 1 describes the blinding function Blind, while the details of the PSS encoding function Encode are described in Appendix D. Once the server receives  $B$ , it signs it using its private key as in a standard RSA signature. Finally, to retrieve a valid signature on  $mHash$ , the client then runs the UnBlind function. Note that to ensure unlinkability, the client must verify that the resulting signature is valid. For completeness, the verification process is described in Appendix A.

The goal of our ZK proof for the blind signature is to prove to the server that the client is requesting a signature for a private token  $t'$ , which is bound to a private identifier  $ID_A$  (which we will also prove is the same identifier bound to the token  $t$  that the client sends). For that, the ZK proof circuit (described in Algorithm 2) takes the blinded message  $B$  sent to the server, and the private witness values required to recompute it – i.e., derive the token  $t'$  from  $ID_A$  and generate the blinded message. It then uses these values to derive the token and recompute the blinded message inside the ZK circuit and verifies that the result, which we denote as  $\hat{B}$ , is indeed equal to  $B$ . From the derived token  $t'$  and the optional private salt value  $salt$ , the circuit computes  $em = \text{Encode}(t', salt)$ , which is the message to be signed. From the private input  $r$ , it re-compute the blinding value  $SK_B = r^e \bmod N$ . Finally, it re-computes the blinded message  $\hat{B} = em \cdot SK_B \bmod N$  and verifies it is indeed the one that was given as a public input. For efficient implementation of modular arithmetic in the circuit, we use the technique from [59].

The above computation proves that the blinded message  $B$  was correctly generated using a token that was derived from  $ID_A$ . As we proved that the blinded message was computed correctly, this means that after getting the server's response, the client will be able to unblind the signature and will hold a valid signature for the token  $t'$ . The “one-more-unforgeability” of the blind signature scheme [48] states that one run of the blind signature protocol can produce a valid signature on at most one message. As we prove that the client can get a valid signature on  $t'$ , this means that it cannot get a valid signature on any other token. As we proved the token was derived from  $ID_A$ , as long as the hash function is collision resistant, it commits to  $ID_A$ , and is bound only to it.

Furthermore, the composition with the ZK proofs maintains the blindness property of the blind signature. The blindness property is defined for messages chosen by the adversary [48], thus the fact that the ZK proof allows the server to learn a predicate on the input does not affect the blindness of the signature. Moreover, as the ZK proof does not leak any information about its witnesses, it does not leak any information about the blinding value  $N_{t'}$ . As  $N_{t'}$  is uniformly random and unknown to the server, the distribution of the tokens for a specific ID value is computationally indistinguishable to the server

---

<sup>3</sup>Note that while the RFC includes the computation of the hash on the message, it explicitly allows computing the hash outside the signature scheme as we do.

---

**Algorithm 2** ZKP circuit for the unlinkable version of 3PaaS. The function takes as input a public and a private witness, as described in Section 5.4.

---

```

function 3PaaS( $(t, B, H, RT)$ ,  $(r, N_t, N_{t'}, ID_A, ID_B, N_H, b, salt)$ ):
   $\tilde{t} \leftarrow h(N_t \parallel ID_A)$ 
  // verify that  $t \leftarrow h(N_t \parallel ID_A)$ 
  assert ( $t = \tilde{t}$ )
   $t' \leftarrow h(N_{t'} \parallel ID_A)$ 
   $em \leftarrow \text{Encode}(t', salt)$ 
   $\tilde{B} \leftarrow (em \cdot r^e) \bmod N$ 
  // verify that  $B \leftarrow \text{Blind}(h(N_{t'} \parallel ID_A); SK_B)$ 
  assert ( $B = \tilde{B}$ )
  if  $b = 1$  then
     $\tilde{H} = h(N_H \parallel ID_A \parallel ID_B)$ 
  else if  $b = 0$  then
     $\tilde{H} = h(N_H \parallel ID_B \parallel ID_A)$ 
  // verify that  $H \in \{h(N_H \parallel ID_A \parallel ID_B), h(N_H \parallel ID_B \parallel ID_A)\}$ 
  assert ( $H = \tilde{H}$ )
  // verify that  $ID_A$  is not revoked (see Section 5.3)
  assert ( $\text{NotRevoked}(ID_A, RT)$ )
  return 1

```

---

from tokens for other ID values or even random tokens.

### 5.3 Proof of non-revocation

As part of our protocol, we want to prove that the identifier  $ID_A$  we used was not revoked, without revealing it to the server. For this, we use a proof of non-membership using a Merkle tree [40], which we implement in our ZK proof as was done by [37]. We assume that the server has a list of all revoked identifiers, and it builds and publishes a binary Merkle tree  $RT$ , where the leaves represent revoked identifiers in a lexicographic order. To prove that a client has a token which is *not* in the revocation list, the client must convince the prover that there exists two adjacent identifiers in the Merkle tree such that its identifier is lexicographically in between them. To allow for a large revocation list, in our zkSNARK proof implementation, we use a Merkle tree of height 30 that can support  $\approx 2^{30}$  revoked tokens (note that we can significantly reduce the required size of the revocation list using key periods, as we describe in Section 7.3).

The proof of non-membership requires two full Merkle paths (each of length 30 in our case), as in the worst case, the two adjacent identifiers will be in two separate subtrees, with no overlap except of the root  $RT_R$ . This requires a ZK proof over a total of 60 calls to the hash function. We propose an optimization that can reduce the cost by a factor of almost two. We partition the leafs of the tree into smaller subtrees (e.g., of size  $2^8$ ), where the first leaf of each tree is the copy of the last leaf of the previous tree. While adding only a small overhead to the number of leafs (less than 0.5% in our case), we now ensure that any two adjacent identifiers we use for our proof are in the same small subtree, and share the same Merkle path from the subtree to the root. This means that instead of  $30 \cdot 2 = 60$  hash computations, our new proof will only require  $8 \cdot 2 + (30 - 8) = 38$  hash computations. Full details of the implementation can be found in Appendix B.

Note that since the Merkle tree is assumed to be large, it might not be practical for the client to download the full revocation list, and compute the full Merkle tree and the Merkle paths for the root required for the proof. To overcome this limitation, the client can request the server to provide the required Merkle paths and use only the paths and the published root of the tree  $RT_R$  in the proof. As this request can be done periodically outside our protocol (e.g., Firefox CRLite updates every 12 hours [55]), it does not leak sensitive private information about new sessions to the server.

### 5.4 Circuit computation for unlinkability

Algorithm 2 shows our ZK proof circuit that is used to produce proofs for the statements described in Section 5.1. The circuit takes as input the public witness  $(t, B, H, RT)$  and the private witness  $(r, N_t, N_{t'}, ID_A, ID_B, N_H, b, salt)$ . The circuit verifies that the public values are indeed computed correctly

Table 1: Benchmark results for ZK Proof generation and verification.

| Device         | Prover time (ms)       |                |            |       | Verification time (ms) |
|----------------|------------------------|----------------|------------|-------|------------------------|
|                | Offline Pre-processing |                | Online     | Total |                        |
|                | Derive ( $t, B$ )      | Non-revocation | Derive $H$ |       |                        |
| MacBook Air M2 | 482                    | 275            | 26         | 783   | 17                     |
| Raspberry Pi   | 2760                   | 1701           | 157        | 4618  | 80                     |

(i.e., are all bounded to the same  $ID_A$ ), and that the identifier  $ID_A$  is not in the revocation list.

First, to check if the public value  $t$  is indeed bounded to  $ID_A$ , the circuit re-derives it using  $(N_t, ID_A)$  and verifies the result is equal to the public input  $t$  (we do not need to prove the validity of the signature  $S$  on the token as it is verified by the server outside the ZK proof).

Second, as described in Section 5.2, we verify that public value  $B$  has been computed correctly, thus proving the resulting new token  $t'$  is bounded to the same  $ID_A$ .

Third, to verify that the public value  $H$  is valid, i.e., bounded to the same  $ID_A$ , the circuit either computes  $h(N_H \parallel ID_A \parallel ID_B)$  or  $h(N_H \parallel ID_B \parallel ID_A)$  based on a private witness bit  $b$ .

Finally, we prove that  $ID_A$  is not revoked, i.e., not a member of the Merkle tree  $RT$ , using the logic described in Section 5.3.

## 5.5 Preprocessing of ZK proofs

We note that although our optimized ZK proof is efficient and practical (see benchmark results in Section 5.6), we can further reduce the latency and run time of the proof generation using preprocessing and proof randomization. We can partition the ZK generation proof into three parts:

1. **One-time offline proof** – the proof for non-revocation is independent of the protocol and the computation can be preprocessed. Moreover, as long as the revocation list is not updated, the same proof for non-revocation can be reused, if the zkSNARK framework we use supports efficient re-randomization of proofs (which is required to maintain unlinkability).
2. **Offline proof** – the proofs for the correct generation of the old and new token (including the blinded message for signature) are independent of the details of the new session we want to open (that include the identifier of the remote party) and thus can also be preprocessed. Moreover, we can precompute this part of the proof for several tokens in the future.
3. **Online proof** – the only part that cannot be preprocessed is the statement that computes  $H$ , as its value is dependent also on identifier  $ID_B$  of the remote party which is not known in advance.

The client can use the preprocessing to reduce latency and re-randomization to reduce the total proof generation time. During the online phase, the different parts of the proof can be composed using a commitment on the private input  $ID_A$ , that ensures it is the same across all proof statements. We instantiate the commitment scheme using Poseidon [36], with a randomness chosen by the client.

**Limitations** The Mirage [41] zkSNARK back-end we use currently does not support proof re-randomization, so in our implementation we re-compute the proof of non-revocation in each run of the protocol.

## 5.6 Benchmark results

We use Circ [54] toolchain to write our ZKP programs (described in Algorithm 2) in front-end language called Z# [54] and compile it to R1CS, using Mirage [41] for the zkSNARK back-end. We further use Woo. et al’s [59] work for expressing modular arithmetic efficiently in R1CS. To benchmark the implementation, we benchmark our ZKP on two client machines. One is a Apple MacBook Air M2 with 24 GB RAM whereas the other is a lesser powered Raspberry Pi with an AMD Cortex-A76 with 8 GB RAM. We present results of the optimized statements using the partitioning to the three parts described in Section 5.5. For test data generation, we use a Rust implementation of RSA blind signature [30], which we modified to use Poseidon as the underlying hash function.

Table 1 shows the benchmark results by instantiating 3PaaS with Poseidon [36], a zkSNARK-friendly hash function. The non-revocation proof also adopts Poseidon-based Merkle tree. Recall that our ZKP consists of two phases. In the first phase proofs can be preprocessed offline during the device’s ideal time. In the next phase, the client computes a new proof online during communication. Furthermore,

the offline phase consists of two proofs: one proves that token  $t$  and blinded value  $B$  are well form and the other is a proof of non-revocation. On the other hand, the online proof consists only of one proof that prove that  $H$  has been correctly derived.

Even on the Raspberry Pi (which is relatively slow even compared to 2019 smartphones [23]), the time to generate the path critical online proof is only 157 milliseconds, while the expensive proofs can all be preprocessed offline.

Mirage produces a constant proof of size 240 bytes. The proof generation requires the client to be provided with a prover key (that is circuit specific and can be reused for all proof generation). The size of the prover key depends on the size of the statement. For example, the circuit that proves that  $(t, B)$  is well-formed is the largest circuit which consists of 30K R1CS constraints and produces a 98 MB prover key. On the other hand, the circuit that proves that  $H$  is correctly formed consists of only 505 constraints and produces a 0.8 MB prover key. The remaining circuit is the one that verifies non-revocation. It consists of 13K constraints and produces a 20 MB prover key. Recall, that we assume the prover keys would be provided to the client during the initial application sign-up process, or as part of a software update.

Our benchmarks suggest that our protocol is practical for real-world deployment. We note that the ZK proof generation dominates the protocol running time. Generating an RSA signature takes less than 1 millisecond, and even a long communication Round Trip Time (RTT), such as from the United States to Australia, usually takes less than 250 milliseconds. Based on the analysis from TokenWeaver [25], the server’s storage requirements for  $10^8$  user are less than 200 GB, while the clients’ main storage costs are the proving keys (20 MB). The communication costs are also negligible as each proof size is only 240 bytes.

## 6 Formal Analysis

In order to confirm that the proposed protocol provides the required security and privacy guarantees, we perform a comprehensive formal analysis using two state-of-the-art verification tools. Namely, we use the TAMARIN prover [56] to analyze security properties, and the DeepSec prover [18, 17] to analyze privacy properties. Both of these tools operate in the *symbolic model of cryptography*, in which messages are represented as abstract terms, and cryptographic functions as equational theories operating on them. These abstractions enable large-scale automated analysis of protocol executions in the presence of a network adversary. For example, TAMARIN implements the standard Dolev-Yao [32] model, which represents an active adversary capable of intercepting, reading, modifying, injecting, and deleting messages.

We briefly introduce both tools and explain how we use them to model and analyze the core properties of our protocol. All of our models are available at [29].

### 6.1 Verification of security properties

The TAMARIN prover [56] is a state-of-the-art verification tool for the automated analysis of security protocols in the symbolic model of cryptography. Given a protocol model and a property, the tool works through backwards reasoning to produce a counterexample representing an attack, or, if one does not exist, conclude that the property holds. Protocols are modeled as collections of labelled multiset rewriting rules defining transitions in the global set of *facts*. Facts are predicate symbols applied to zero or more terms that represent the global state. A rewriting rule  $r : p \multimap a \rightarrow c$  consists of three parts: a premise ( $p$ ), a set of actions ( $a$ ), and a conclusion ( $c$ ), each containing zero or more multisets of facts. When a rule is executed, the facts in the premise are replaced with the ones in the conclusion and the transition is marked with the set of action facts.

In the symbolic model, cryptographic primitives are commonly expressed as *function symbols* that consist of zero or more terms. Each function has an *arity*, which determines the number of terms it contains. For example, we can define a pair of functions,  $senc/2$  and  $sdec/2$ , to denote symmetric encryption and decryption, respectively. Furthermore, to model the correct behavior of the operations, we define an *equational theory* that specifies their equivalence properties. In the case of symmetric encryption, it states that if we have the key that a message was encrypted with, we can use it to decrypt the original message:  $sdec(senc(m, k), k) = m$ .

In the linkable protocol, we only require two cryptographic primitives: a one-way hash function and digital signatures, which have built-in definitions in TAMARIN. In the unlinkable protocol, however, we also use blind signatures and zero-knowledge proofs. For both of these, we need to define custom functions and equational theories that correspond to the properties we assume for the protocol. In particular, the

---

```

1 lemma pcs:
2   " All p c b s t id n #i.
3     /* If a server accepts a token t, */
4     AcceptToken(p, c, b, s, t)@i & t = h(<n, id>)
5     ==> /* then either */
6       ( /* (1) a client sent the token, or */
7         Ex #j. SendToken(c, id, b, s, t)@j & j < i
8       ) | (
9         /* (2) the server was compromised, or */
10        Ex u #j. Compromise(p, <'server_keys', u>)@j & j < i
11      ) | (
12        /* (3) the client was compromised.. */
13        Ex t2 #j. Compromise(id, <'client_token', t2>)@j & j < i
14        /* ..and did not heal.. */
15        & (Ex b2 s2 #k. AcceptToken(p, 'att', b2, s2, t2)@k)
16        /* ..or revoke its identifier. */
17        & not (Ex #k. Revoke(id, p)@k & k < i)
18      )
19   "

```

---

Listing 1: PCS property in the Tamarin models

definition for zero-knowledge proofs must encode the exact set of terms that are expected for the proof to hold. As is the norm in symbolic verification, the model treats cryptographic primitives as perfect operations that do not leak any information.

Protocol properties (*lemmas*) are expressed as fragments of first-order logic extended with timepoints. Lemmas encode the expected protocol behavior by referring to action facts in the global trace. For example, we can model weak agreement of a message  $m$  between a sender  $s$  and a receiver  $r$  as follows:

```

lemma weak_agreement:
  " All m s r #t1. Receive(r, s, m)@t1
    ==> Ex m2 #t2. Send(s, r, m2)@t2 & t2 < t1 "

```

Informally, this states that whenever  $r$  receives a message  $m$  at a timepoint  $\#t1$ , allegedly from  $s$ , there must exist an earlier timepoint  $\#t2$  at which  $s$  sent a message to  $r$ . Since we only consider *weak* agreement, we allow the sent message  $m2$  to be different than the received one.

In our model, we assume that there is a secure connection (over e.g., TLS) between each client and server, which the adversary cannot intercept. However, we allow the adversary to compromise individual clients and server, and to initiate its own secure channel with non-compromised entities. Whenever it reads data from the channel or injects messages into it, the model produces a corresponding action fact (*Leak* or *Inject*) and uses the channel identifier *'att'*.

We model the PCS property (see Listing 1) as follows: If a PCS server accepts a token  $t$  at timepoint  $\#i$ , then either (1) there exists a client that sent it at a previous timepoint  $\#j < \#i$ ; or (2) the server's long-term keys were leaked at a previous timepoint  $\#j < \#i$ ; or (3) an adversary compromised the client at a previous timepoint  $\#j < \#i$ , s.t., the client did not heal or revoke its identifier before the adversary consumed its token. In the third case, the adversary deprecates the honest client's token, preventing it from performing the healing step. When that happens, the client needs to revoke its identifier and derive a new token out of band.

We model ZKPs as perfect cryptographic functions, which can be verified by the server without revealing any sensitive data about the private witnesses. After receiving an *AuthenticationAndTokenQuery* (see Figure 2), the server first checks that the client provided a valid (blind) signature for the old token  $t$ . Then, as described in Section 4.3, it uses the provided ZKP to verify the statements described in Section 5.1, i.e., that (1) the same identifier  $ID$  was used to construct  $t$ ,  $t'$ , and  $H$ , (2)  $B$  is a blinding of the token  $t'$ , which was constructed from identifier  $ID$  (and a nonce), and (3) the identifier  $ID$  is not in the public Merkle tree of revoked identifiers. The server learns nothing from the check, beyond the fact that it either succeeds or fails.

To help TAMARIN avoid loops and terminate faster, we define several helper lemmas that the tool can reuse in the more complicated proofs. We also make use of *restrictions*, which are a built-in feature that allows us to break loops and reject traces that lead to uninteresting results. For example, we can restrict the model to only compromise each token once, since doing so repeatedly does not increase the adversary's

knowledge. Finally, for lemmas that are unable to terminate automatically or are unreasonably slow, we write custom *tactics*, which allow us to guide the verification process by defining custom proof tactics for each lemma.

We model both versions of our protocol. TAMARIN is able to prove that (1) they both provide post-compromise security in the presence of an active network adversary, (2) the adversary is unable to spoof signatures without compromising the server’s long-term keys, and (3) the adversary cannot spend tokens that have been revoked. Additionally, we also run several sanity checks to ensure that our models correctly capture the details of the protocols. With the help of the optimization strategies discussed in the previous paragraph, TAMARIN is able to prove all lemmas in a matter of minutes, making the results easily reproducible on an average laptop or desktop computer. However, we note that optimizing the models to be this efficient required significant manual effort.

## 6.2 Verification of privacy properties

Modern verification tools for the symbolic modeling of cryptographic protocols, such as TAMARIN [56] and ProVerif [9], predominantly focus on verifying *trace properties*, which are ideal for modeling security attributes. For example, as shown previously, we can express secrecy and authentication properties as fragments of first-order logic. Using state-of-the-art search algorithms, these tools are able to efficiently verify trace properties in an unbound state space. However, in contrast, most of them only offer limited support for analyzing coarse-grained *equivalence properties*, which are necessary to prove anonymity and privacy guarantees.

Observational equivalence refers to the indistinguishability of two systems from an outsider’s perspective. In order to verify that our proposed protocol properly hides the identities of the clients, we need to show that an outsider (including a malicious PCS server), is unable to distinguish between two ‘healing’ steps and therefore cannot link them to any specific user. For this, we use the DeepSec prover [18, 17].

DeepSec is a state-of-the-art verification tool intended for analyzing privacy properties of cryptographic protocols. In particular, it provides an automated way of determining trace equivalence in a bound number of parallel sessions. The tool takes as input a protocol model, written in applied pi calculus, and either confirms anonymity, or returns a trace representing an attack. Protocols are modeled as two or more processes, which are executed in parallel.

We use DeepSec to verify that the unlinkable version of our protocol provides the expected privacy properties. Specifically, we show that since the zero-knowledge proof hides all identifiers, even a malicious PCS server is unable to distinguish between different sessions. We model this by letting the adversary play the role of a compromised server that communicates with honest clients. Furthermore, we model the possibility of collusion between the server and malicious clients. After an honest client  $A$  requests a credential to open a new session with a malicious client  $C$ ,  $C$  can then leak  $A$ ’s identity to the server and link the specific interaction with the server (e.g., by comparing the values of  $H$  and  $M$ ). Thus, we model this collusion by revealing the honest party identifier to the server. Our model proves that even after such collusion, a malicious server cannot link these identified interactions with other interactions involving only honest clients that do not collude with the server.

We model two cases of interaction between honest clients and a malicious server (see Listing 2). In both cases, two instances of client  $A$  and two instances of client  $B$  interact with the server s.t., they reveal their identifiers at the end of each interaction. Then, in case 1, client  $A$  interacts with the server an additional time without revealing its identifier and  $B$  does nothing. In case 2, the roles are reversed. The query checks whether the malicious server is able to distinguish between the two cases, i.e., whether it can use the initial interactions to deduce which client made the anonymized request. All processes are executed in parallel and can be completed in any order. Thus, the two “revealed” instances of each client can model the case that both the malicious server can learn the identities (and the spent tokens) in the interactions, both before and after the target interaction. In both cases, the server is allowed to interact with a third, malicious client that reveals all secrets in each session.

Before proving that our unlinkable protocol provides the expected privacy properties, we model the linkable version as a sanity check. As expected, DeepSec is able to instantly find a counterexample showing that the compromised PCS server can distinguish between two incoming signing requests, since they contain the identifier of the sender in plaintext. In the unlinkable version, however, the same attack is not possible, even when increasing the number of parallel sessions and signing requests.

---

```

1 let Case_1 =
2   new pid;
3   ( Init(pid)                (* Initialize clients *)
4     | (!^2 PA(pid, true))    (* Client A interacts with the server twice *)
5     | (!^2 PB(pid, true))    (* Client B interacts with the server twice *)
6     | (!^1 PC(pid, true))    (* Client C interacts with the server *)
7     | (!^1 PA(pid, false))   (* Client A interacts with the server *)
8   )
9 .
10
11 let Case_2 =
12   new pid;
13   ( Init(pid)                (* Initialize clients *)
14     | (!^2 PA(pid, true))    (* Client A interacts with the server twice *)
15     | (!^2 PB(pid, true))    (* Client B interacts with the server twice *)
16     | (!^1 PC(pid, true))    (* Client C interacts with the server *)
17     | (!^1 PB(pid, false))   (* Client B interacts with the server *)
18   )
19 .
20
21 (* Check if a malicious server is able to determine if it is interacting
22    with Client A or Client B after learning the session identifiers of
23    both A and B in previous interactions. *)
24 query trace_equiv(Case_1, Case_2).

```

---

Listing 2: Unlinkability check in the DeepSec models

## 7 Discussion

In this section, we discuss potential applications for 3PaaS and give an example of how it can be integrated into secure messaging. As a case study, we use Signal and the X3DH [50] protocol and discuss practical implementation details. We also discuss how we circumvent impossibility results, and outline future work.

### 7.1 Applications for Signal

We note that many current messaging protocols, such as Signal [50] and MLS [4], rely on long-term identity public keys to (implicitly) authenticate outgoing messages. If the keys are compromised, they lose security against active network adversaries. Our solution can be adapted to address this issue while minimizing changes (and possibly even maintaining backward compatibility) to the original protocols.

As a concrete example, we briefly outline how 3PaaS can be adapted to Signal and the X3DH [50] protocol. As in the vanilla X3DH, when starting a new session, the sender retrieves the receiver’s signed pre-key  $SPK_B$ , and an optional one-time pre-key  $OPK_B$ . It then generates an ephemeral key pair with the public key  $EK_A$ . In our protocol, it will then request the PCS server for a new credential attesting the hash of the first message it intends to send, and both identifiers ( $ID_A$  and  $ID_B$ ). The PCS server checks the validity of the token and the ZK proofs and responds with (1) a credential for the message and the identifiers, and (2) the blind signature on the new token. When sending the first message, the client encrypts the credential with a key that is derived from the shared session key and appends it to the message. The message flow of the full combined protocol is shown in Figure 3

#### 7.1.1 PCS for sessions

While the original Signal protocol can only provide PCS for keys during a single session, our modified protocol can also provide PCS to newly spawned sessions. However, due to the asynchronous nature of the communication, the exact PCS guarantees may depend on the exact nature of the compromise and specific implementation details. If the adversary can only compromise the long-term identity keys and the current PCS token, we will get PCS for any new session. This is because the adversary needs to request a new credential to open a new session.

While a complete formalization and proof of the combined protocol is out of scope for this paper, we use the TAMARIN prover to model a simplified version of the X3DH key exchange protocol and verify that combining it with 3PaaS achieves PCS across sessions. The message flow of the simplified version is shown in Figure 4. Message requests are only accepted if the initiator is able to provide a valid signature

from the PCS server, which requires an unspent token. Clients are able to heal from a compromise by spending the leaked token before the adversary. If, however, the adversary spends the token of a compromised client before the healing can take place, the client will detect it when trying to initiate a new session and can heal out of band by revoking its identifier, as described in Section 4.3.

Using the model, we prove that the following PCS property holds: If client  $B$  accepts a message from client  $A$  at timepoint  $i$ , then either (1) client  $A$  sent the message at a previous timepoint  $j < i$ ; or (2) the PCS server’s long-term key was leaked at a previous timepoint  $j < i$ ; or (3) client  $A$ ’s token was leaked at a previous timepoint  $j < i$ , s.t., it did not heal or revoke its identifier before the token was consumed by the adversary.

We also prove that including the identifiers of both endpoints in the zero-knowledge proof is necessary to achieve the required security properties. First, we show that removing both of the identifiers ( $ID_A$  and  $ID_B$ ) allows an adversary to use a token from a compromised client to request signatures for sessions between any two clients. For example, client  $C$  can open a new session between client  $A$  and client  $B$ . Second, we show that omitting only the responder’s identifier ( $ID_B$ ) allows the same signature to be used in multiple sessions. For example, client  $C$  can use the same signature to open two separate sessions with client  $A$  and client  $B$ .

For the specific remote parties that the adversary requested credentials for, PCS will only be regained when all of these credentials are not usable anymore. Although these credentials can be short-lived, we need to support the maximum time a message can be stored in the server queue before delivery, which can be as long as 45 days in the case of Signal<sup>4</sup>. One option to overcome this limitation is if the messaging server (i.e., Signal) can provide a signed timestamp for when the message was delivered. In that case, we can use very short-lived credentials (e.g., life span of several minutes), where the validity of the credential will be checked against the time the message was delivered using the Signal server timestamp.

We note that a compromised timestamping server could collude with a malicious client to break PCS. The client could use old expired credentials, but ask the server to sign the messages with a timestamp from the past. To mitigate this, we can use multiple timestamping servers to increase trust in the system.

### 7.1.2 Privacy and deniability

Since 3PaaS is unlinkable and does not reveal either the sender’s or the receiver’s identity, using it does not affect the users’ privacy. We note that Signal supports a notion of *offline deniability* [50]. As mentioned in Section 4.2.1, 3PaaS was designed with such notions in mind. However, due to the complexity and subtleties of Signal’s deniability properties [58, 39, 34, 22], we leave the formal definition and analysis of the deniability properties of the combined protocol for future work.

## 7.2 Relation to the impossibility results of [27]

One of the main implications of [27] is that protocols that need to be robust (i.e., need to seamlessly continue communicating) even after loss of dynamic state (i.e., restoring system backups, getting a new phone, etc.) fundamentally cannot achieve PCS. Informally, the argument is that PCS requires that obtaining old participant states should not enable communication in the future. Instead, communication should require knowing the most recent state.

One generic way to sidestep this impossibility result is to drop the robustness requirement and exploit some out-of-band communication to resynchronize after dynamic state loss occurs. However, in all previous PCS solutions, the PCS dynamic state is strictly handled between each pair of participants. Thus, if a party  $A$  loses its dynamic state, it would have to resynchronize out-of-band with each of its contacts (e.g., requiring a messaging app user to perform some out-of-band resynchronize with each and every contact they communicated with in the past). This is an infeasible approach in practice for all but the most security-critical environments.

In contrast, our solution also uses out-of-band for this scenario, but because we leverage a server to establish PCS, dynamic state loss can be repaired through a single out-of-band process with just the server. This has two major benefits: First, there is no need to communicate out-of-band with any of the contacts. Second, in practice, out-of-band channels with central servers already exist for most services (such as messengers, banking services, healthcare providers, etc.), exactly to cater for cases like lost or broken devices. In fact, all major End-to-End-Encrypted messaging apps (e.g., WhatsApp and Signal) already require such out-of-band communication to recover from state loss with the server (e.g., if a user’s phone breaks down or is stolen). Such out-of-band channels can include the knowledge of a secret recovery

<sup>4</sup><https://signal.org/blog/a-synchronized-start-for-linked-devices/>

PIN, one-time codes sent using SMS messages, and even more complicated processes that require the user to prove their real identity using official government-provided documentation. We can thus leverage those existing processes to also repair after dynamic state loss without adding new assumptions to the system.

While such out-of-band recovery processes are widely deployed in practice and usually play a critical role in the security of the system, they are not usually formalized and modeled. While such formalization is outside the scope of this current work, we believe that a better analysis of such protocols and how they should be formalized as part of security proofs is an important open problem that should be addressed in future work.

### 7.3 Limitations and future work

As future work, we envision additional case studies and implementations. As part of this process, several practical details need to be considered for each use case separately. Here, we outline some of the most important ones and propose future extensions to 3PaaS.

**Subversion-resistant zero-knowledge proofs and post-quantum support** One main limitation of our work is that the Mirage [41] framework we used in our benchmark implementation is not “Zero-knowledge Subversion-resistant”. This means that the party that generates the CRS, i.e., the parameters that are used to instantiate the ZK proof algorithm, may maliciously generate CRS values in a way that will break the ZK property and leak private information. One possible mitigation that is widely used by cryptocurrencies is to generate these parameters in a “ceremony” using a secure multiparty computation [6, 11, 12], such that as long as there is at least one honest party, the resulting parameters will be safe to use. Apart from the one-time cost of running the ceremony, there is no added overhead to the protocol. Another mitigation option is to augment the CRS generation in a way that will allow the users to verify that it was honestly generated [1]. For the protocol, this only adds a one-time cost of verifying the CRS values (which can be done once on the publicly published CRS values), and a negligible storage overhead (one group element or several hundred bits). A third option is to switch to a ZK proof system that is unconditionally ZK Subversion-resistant.

Another limitation is that the blind signatures and zero-knowledge proofs we use are not quantum-resistance. In recent years, researchers have proposed post-quantum versions of both [42, 3, 47, 13]. Therefore, future versions of the protocol could achieve quantum-resistance.

**Key-based revocation** We currently implement revocation by deprecating identifiers over an out-of-band channel, as outlined in Section 4.3. In addition to requiring users to maintain a secure connection with the service provider, it also has the drawback of making the revoked identifiers public (as they are published in the Merkle tree) and forcing users to replace their identifiers after they have been revoked. Future extensions of the protocol could instead use a revocation key, which would be included together with the public identifier in the token generation and checked in the ZKProofs. This does not leak the client’s identifier in the revocation list and allows it to be reused in the future with a different revocation key.

**PCS for the receiving client** Our current solution provides PCS to the sender, however, it does not guarantee PCS if the receiving client was compromised. In addition to the receiver’s long-term identity key, Signal’s X3DH protocol handshake also uses the receiver’s “prekeys”, which include a signed short term receiver key that is rotated periodically by the receiver, and optional one-time prekeys. Our 3PaaS can be extended to provide PCS to this “prekey bundle”. Although formalization and full analysis of the exact PCS guarantees are outside the scope of this work, intuitively, adding 3PaaS signatures for these keys can allow the receiving client to heal each time these keys are rotated. PCS for the receiver can also be achieved by adding a 3PaaS signature to the first response message in each new session.

**PCS for broadcast messages** Our solution could also be extended to provide PCS for broadcast messages sent in group chats. The PCS sender will use ZK proof to verify that the message sender’s identity is the same as the one bound to the token, and sign the broadcast message.

**Supporting key periods via linkable chains** In our current solution, the server needs to store the list of used tokens and the list of revoked identifiers, both of which increase in size over time. To avoid this, we can incorporate the linkable chain idea from TokenWeaver to provide support for *key periods*, which will cause all the signed tokens to expire after some predefined time period. The user can periodically use

a linkable protocol to receive a new anonymous token for the new chain. This allows the server to clear its public revocation list, improve the efficiency of the proof, and safely delete all the previous periods' spent tokens.

**Unlinkable channel assumption** Similar to TokenWeaver [25], the unlinkability property of the 3PaaS protocol relies on the availability of an unlinkable communication channel. While this can be solved by using the Tor network or a relay server trusted to hide the communication metadata, these approaches add trust assumptions and might impact performance. Future work should further explore this solution space and the various tradeoffs.

**Exploration of zero-knowledge proof tradeoffs** In our benchmark implementation, we use Mirage [41] for the zero-knowledge proofs. It provides small signatures with a fast verification time at the cost of a longer proof generation time. In future work, we will consider other options with slightly longer verification time that allow for much faster proof generation. Similar tradeoffs can be done with regard to the proof size and the computation time.

**Multiple sessions per token request** In the current solution, each (successful) request to the PCS server returns allows us to open exactly one new session (via the signature on the  $H$  value). In future extensions of the protocol, we could combine multiple requests into one query. As this is a relatively cheap part of the zero-knowledge proof (approximately 5% of the total proof generation time), amortizing ZK proofs for multiple  $H$  values in a single protocol run could significantly reduce the total computational and communication cost, especially in cases where we want to open many new sessions at once, such as right after installing an application.

## 8 Conclusions

We presented 3PaaS, the first privacy-preserving protocol for providing post-compromise security as a service. Unlike current solutions, our protocol works independently of the application it is used for and is, therefore, able to work over multiple sessions, groups, and services. We implemented our solution using zero-knowledge proofs with novel optimizations and formally analyzed it with two state-of-the-art symbolic verification tools, the TAMARIN prover and DeepSec, to prove that it meets the expected efficiency, privacy, and security requirements. Finally, we outlined how the protocol can be adapted to modern secure messaging applications, like Signal, with minimal changes and backward compatibility.

## Acknowledgments

Eyal Ronen was supported in part by an Israel Science Foundation grant no. 1807/23, the Len Blavatnik and the Blavatnik Family Foundation, and the Stellar Development Foundation.

## References

- [1] Behzad Abdolmaleki, Helger Lipmaa, Janno Siim, and Michal Zajac. On subversion-resistant SNARKs. *J. Cryptol.*, 34(3):17, 2021.
- [2] Joël Alwen, Sandro Coretti, and Yevgeniy Dodis. The double ratchet: Security notions, proofs, and modularization for the Signal protocol. In *Advances in Cryptology – EUROCRYPT 2019*, pages 129–158, Cham, 2019. Springer.
- [3] Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. Non-interactive blind signatures: Post-quantum and stronger security. In *Advances in Cryptology – ASIACRYPT 2024*, pages 70–104, Singapore, 2025. Springer.
- [4] Richard Barnes, Benjamin Beurdouche, Raphael Robert, Jon Millican, Emad Omara, and Katriel Cohn-Gordon. The messaging layer security (MLS) protocol. RFC 9420, July 2023.
- [5] Mihir Bellare, Asha Camper Singh, Joseph Jaeger, Maya Nyayapati, and Igors Stepanovs. Ratcheted encryption and key exchange: The security of messaging. In *Advances in Cryptology – CRYPTO 2017*, pages 619–650, Cham, 2017. Springer.

- [6] Eli Ben-Sasson, Alessandro Chiesa, Matthew Green, Eran Tromer, and Madars Virza. Secure sampling of public parameters for succinct zero knowledge proofs. In *IEEE Symposium on Security and Privacy*, pages 287–304. IEEE Computer Society, 2015.
- [7] Alexander Bienstock, Jaiden Fairoze, Sanjam Garg, Pratyay Mukherjee, and Srinivasan Raghuraman. A more complete analysis of the Signal double ratchet algorithm. In *Advances in Cryptology – CRYPTO 2022*, pages 784–813, Cham, 2022. Springer.
- [8] Nina Bindel, Nicolas Gama, Sandra Guasch, and Eyal Ronen. To attest or not to attest, this is the question - provable attestation in FIDO2. In *Advances in Cryptology – ASIACRYPT 2023*, pages 297–328. Springer, 2023.
- [9] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016.
- [10] Olivier Blazy, Angèle Bossuat, Xavier Bultel, Pierre-Alain Fouque, Cristina Onete, and Elena Pagnin. SAID: Reshaping Signal into an identity-based asynchronous messaging protocol with authenticated ratcheting. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 294–309. IEEE, 2019.
- [11] Sean Bowe, Ariel Gabizon, and Matthew D. Green. A multi-party protocol for constructing the public parameters of the pinocchio zk-SNARK. In *Financial Cryptography Workshops, Lecture Notes in Computer Science*, pages 64–77. Springer, 2018.
- [12] Sean Bowe, Ariel Gabizon, and Ian Miers. Scalable multi-party computation for zk-SNARK parameters in the random beacon model. *Cryptology ePrint Archive*, Paper 2017/1050, 2017.
- [13] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS ’17*, pages 1825–1842, New York, NY, USA, 2017. ACM.
- [14] Melissa Chase, Apoorvaa Deshpande, Esha Ghosh, and Harjasleen Malvai. SEEMless: Secure end-to-end encrypted messaging with less trust. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS ’19*, pages 1639–1656, New York, NY, USA, 2019. ACM.
- [15] David Chaum. Blind signatures for untraceable payments. In *Advances in Cryptology*, pages 199–203, Boston, MA, USA, 1983. Springer.
- [16] Brian Chen, Yevgeniy Dodis, Esha Ghosh, Eli Goldin, Balachandar Kesavan, Antonio Marcedone, and Merry Ember Mou. Rotatable zero knowledge sets: Post compromise secure auditable dictionaries with application to key transparency. In *Advances in Cryptology – ASIACRYPT 2022: 28th International Conference on the Theory and Application of Cryptology and Information Security*, pages 547–580, Berlin, Heidelberg, 2022. Springer.
- [17] Vincent Cheval, Steve Kremer, and Itsaka Rakotonirina. DeepSec: Deciding equivalence properties in security protocols theory and practice. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 529–546. IEEE, 2018.
- [18] Vincent Cheval, Steve Kremer, and Itsaka Rakotonirina. The DeepSec prover. In *Computer Aided Verification*, pages 28–36. Springer, 2018.
- [19] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the Signal messaging protocol. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 451–466. IEEE, 2017.
- [20] Katriel Cohn-Gordon, Cas Cremers, and Luke Garratt. On post-compromise security. In *2016 IEEE 29th Computer Security Foundations Symposium*, pages 164–178. IEEE, June 2016.
- [21] Katriel Cohn-Gordon, Cas Cremers, Luke Garratt, Jon Millican, and Kevin Milner. On ends-to-ends encryption: Asynchronous group messaging with strong security guarantees. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS ’18*, pages 1802–1819, New York, NY, USA, 2018. ACM.

- [22] Daniel Collins, Simone Colombo, and Loïs Huguenin-Dumittan. Real-world deniability in messaging. *Proceedings on Privacy Enhancing Technologies*, 1:320–340, 2025.
- [23] Raspberry Pi 5 B (broadcom BCM2712) vs Qualcomm Snapdragon 865 - benchmark, comparison and differences. [https://www.cpu-monkey.com/en/compare\\_cpu-raspberry\\_pi\\_5\\_b\\_broadcom\\_bcm2712-vs-qualcomm\\_snapdragon\\_865](https://www.cpu-monkey.com/en/compare_cpu-raspberry_pi_5_b_broadcom_bcm2712-vs-qualcomm_snapdragon_865), 2025. Accessed: August 26, 2025.
- [24] Cas Cremers, Britta Hale, and Konrad Kohbrok. The complexities of healing in secure group messaging: Why cross-group effects matter. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1847–1864. USENIX Association, August 2021.
- [25] Cas Cremers, Gal Horowitz, Charlie Jacomme, and Eyal Ronen. TokenWeaver: Privacy preserving and post-compromise secure attestation. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4173–4191, Los Alamitos, CA, USA, May 2025. IEEE.
- [26] Cas Cremers, Charlie Jacomme, and Aurora Naska. Formal analysis of session-handling in secure messaging: Lifting security from sessions to conversations. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1235–1252, Anaheim, CA, USA, August 2023. USENIX Association.
- [27] Cas Cremers, Niklas Medinger, and Aurora Naska. Impossibility results for post-compromise security in real-world communication systems. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4391–4405, Los Alamitos, CA, USA, May 2025. IEEE.
- [28] Cas Cremers, Abhinav Nakarmi, Aleksi Peltonen, and Eyal Ronen. Implementation of the ZKP from 3PaaS. <https://github.com/pseudoproof/3PaaS-ZKP-public>, June 2026.
- [29] Cas Cremers, Abhinav Nakarmi, Aleksi Peltonen, and Eyal Ronen. Symbolic models of the 3PSaaS protocol for the Tamarin prover and the DeepSec prover. <https://github.com/peltona/3PaaS>, June 2026.
- [30] Frank Denis. Rust blind rsa signatures. <https://github.com/jedisct1/rust-blind-rsa-signatures>, 2023. Accessed: June 24, 2026.
- [31] Frank Denis, Frederic Jacobs, and Christopher A. Wood. RSA blind signatures. RFC 9474, Oktober 2023.
- [32] Danny Dolev and Andrew Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.
- [33] EU digital markets act (DMA). <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022R1925>, September 2022. Accessed: August 13, 2025.
- [34] Rune Fiedler and Christian Janson. A deniability analysis of Signal’s initial handshake PQXDH. *Proceedings on Privacy Enhancing Technologies*, 4:907–928, 2024.
- [35] Esha Ghosh and Melissa Chase. Weak consistency mode in key transparency: OPTIKS. Cryptology ePrint Archive, Paper 2024/796, 2024.
- [36] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for Zero-Knowledge proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535. USENIX Association, August 2021.
- [37] Paul Grubbs, Arasu Arun, Ye Zhang, Joseph Bonneau, and Michael Walfish. Zero-knowledge middleboxes. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4255–4272, Boston, MA, USA, August 2022. USENIX Association.
- [38] Jose Jaeger and Igors Stepanovs. Optimal channel security against fine-grained state compromise: The safety of messaging. In *Advances in Cryptology – CRYPTO 2018*, pages 33–62, Cham, 2018. Springer.
- [39] Shuichi Katsumata, Guilhem Niot, Ida Tucker, and Thom Wiggers. Comprehensive deniability analysis of Signal handshake protocols: X3DH, PQXDH to fully post-quantum with deniable ring signatures. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6797–6816. USENIX Association, 2025.

- [40] Paul C. Kocher. On certificate revocation and validation. In *Proceedings of the Second International Conference on Financial Cryptography*, FC '98, pages 172–177, Berlin, Heidelberg, 1998. Springer.
- [41] Ahmed Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, and Dawn Song. MIRAGE: Succinct arguments for randomized algorithms with applications to universal zk-SNARKs. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2129–2146. USENIX Association, August 2020.
- [42] Veronika Kuchta, Jason T. LeGrow, and Edoardo Persichetti. Post-quantum blind signatures from Matrix code equivalence. Cryptology ePrint Archive, Paper 2025/274, 2025.
- [43] Julia Len, Melissa Chase, Esha Ghosh, Daniel Jost, Balachandar Kesavan, and Antonio Marcedone. ELEKTRA: Efficient Lightweight multi-dEvice Key TRAnsparency. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, pages 2915–2929, New York, NY, USA, 2023. ACM.
- [44] Julia Len, Melissa Chase, Esha Ghosh, Kim Laine, and Radames Cruz Moreno. OPTIKS: An optimized key transparency system. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4355–4372, Philadelphia, PA, USA, August 2024. USENIX Association.
- [45] Julia Len, Esha Ghosh, Paul Grubbs, and Paul Rösler. Interoperability in end-to-end encrypted messaging. Cryptology ePrint Archive, Paper 2023/386, 2023.
- [46] Felix Linker, Ralf Sasse, and David Basin. A formal analysis of Apple’s iMessage PQ3 protocol. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 5015–5034, Seattle, WA, USA, August 2025. USENIX Association.
- [47] Alex Lombardi, Fermi Ma, and Nicholas Spooner. Post-quantum zero knowledge, revisited or: How to do quantum rewinding undetectably. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 851–859, Los Alamitos, CA, USA, November 2022. IEEE.
- [48] Anna Lysyanskaya. Security analysis of RSA-BSSA. In *Public-Key Cryptography – PKC 2023*, pages 251–280. Springer, 2023.
- [49] Harjasleen Malvai, Lefteris Kokoris-Kogias, Alberto Sonnino, Esha Ghosh, Ercan Oztürk, Kevin Lewi, and Sean Lawlor. Parakeet: Practical key transparency for end-to-end encrypted messaging. In *30th Annual Network and Distributed System Security Symposium, NDSS*. The Internet Society, January 2023.
- [50] Moxie Marlinspike and Trevor Perrin. The X3DH key agreement protocol, November 2016. Revision 1.
- [51] Brendan McMillion. Key transparency architecture. Internet-Draft draft-ietf-keytrans-architecture-03, Internet Engineering Task Force, February 2025. Work in Progress.
- [52] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398, Washington, D.C., USA, August 2015. USENIX Association.
- [53] Kathleen Moriarty, Burt Kaliski, Jakob Jonsson, and Andreas Rusch. PKCS #1: RSA cryptography specifications version 2.2. RFC 8017, November 2016.
- [54] Alex Ozdemir, Fraser Brown, and Riad S. Wahby. CirC: Compiler infrastructure for proof systems, software verification, and more. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2248–2266. IEEE, 2022.
- [55] John Schanck. CRLite: Fast, private, and comprehensive certificate revocation checking in Firefox. <https://hacks.mozilla.org/2025/08/crlite-fast-private-and-comprehensive-certificate-revocation-checking-in-firefox/>, August 2025. Accessed: August 26, 2025.
- [56] Benedikt Schmidt, Simon Meier, Cas Cremers, and David Basin. Automated analysis of Diffie-Hellman protocols and advanced security properties. In *2012 IEEE 25th Computer Security Foundations Symposium*, pages 78–94. IEEE, 2012.

- [57] Douglas Stebila. Security analysis of the iMessage PQ3 protocol. Cryptology ePrint Archive, Paper 2024/357, 2024.
- [58] Nihal Vatandas, Rosario Gennaro, Bertrand Ithurburn, and Hugo Krawczyk. On the cryptographic deniability of the Signal protocol. In *Applied Cryptography and Network Security*, pages 188–209, Cham, 2020. Springer.
- [59] Anna Pui Yung Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3291–3308, Los Alamitos, CA, USA, May 2025. IEEE.
- [60] zkp application. circom-rsa-verify. <https://github.com/zkp-application/circom-rsa-verify>, 2023. Accessed: April 14, 2025.

## A Blind RSA Signature Verification

Algorithm 3 shows the Blind RSA signature verification process using the PSS verification algorithm `Verify`. We use the same notation as described in the paper. The algorithm takes as input a message  $mHash$ , a signature  $\sigma$ , and a public key  $(e, N)$ . The algorithm outputs *accept* if the signature is valid, else  $\perp$ .

## B Optimized Non-membership Proofs

For our benchmark, we insert  $2^{30}$  identifiers in the Merkle tree of revoked users. To verify the non-membership of a given identifier, as explained in Section 5.3, the verifier has to check that two adjacent identifiers are in the tree, which results in computing 60 individual nodes. To make the process more efficient, we optimize the number of node computations by generating a set of sub-trees instead.

We generate  $2^{22}$  sub-trees of size  $2^8$ . Each sub-tree is identified by a unique lexicographic bound. For consistency, we make the last identifier of each subtree to be the first identifier of its next sub-tree, leaving us with slightly less than  $2^{30}$  identifiers. To prove non-membership of an identifier, the prover can convince the verifier that there exists two adjacent identifiers in one of the sub-trees (without revealing which one), such that their identifier is lexicographically between the two. We prove that an identifier is not in a subtree of size  $2^8$  and that this sub-tree is one of the  $2^{22}$  subtrees. This can be done by computing 38 nodes instead of 60.

## C Application Example: X3DH

Figure 3 shows the full protocol execution of X3DH [50] with the unlinkable version of 3PaaS. Figure 4 replaces X3DH with a simplified protocol that provides the same PCS guarantees.

---

**Algorithm 3** Blind RSA Signature Verification.

---

**function** SigVer( $(mHash, \sigma), (e, N)$ ):

$em \leftarrow \sigma^e \bmod N$

**return** Verify( $mHash, em$ )

**function** Verify( $mHash, em$ ):

**if**  $emLen < hLen + sLen + 2$  **then**

**return**  $\perp$

*// verify that the last octet of em is 0xBC*

$maskedDB \leftarrow \{ \text{leftmost } (emLen - hLen - 1) \text{ octet of } em \}$

*// verify that the leftmost  $(8 \cdot emLen - emBits)$  bits of the leftmost octet in maskedDB are zero*

$S \leftarrow \{ \text{the next } hLen \text{ octets} \}$

$dbMask \leftarrow \text{MGF}(S, emLen - hLen - 1)$

$DB \leftarrow \{ maskedDB \oplus dbMask \}$

*// set the leftmost  $(8 \cdot emLen - emBits)$  bits of the leftmost octet in DB to zero*

*// verify that  $emLen - hLen - sLen - 2$  leftmost octets of DB are zeros*

*// verify that the octet at position  $emLen - hLen - sLen - 1$  of DB is 0x01*

$salt \leftarrow \{ \text{the last } sLen \text{ octets of } DB \}$

$M' \leftarrow \{(0x) 00 00 00 00 00 00 00 00 \parallel mHash \parallel salt\}$

$mHash' \leftarrow h(M')$

**if**  $mHash = mHash'$  **then**

**return** *accept*

**else**

**return**  $\perp$

---

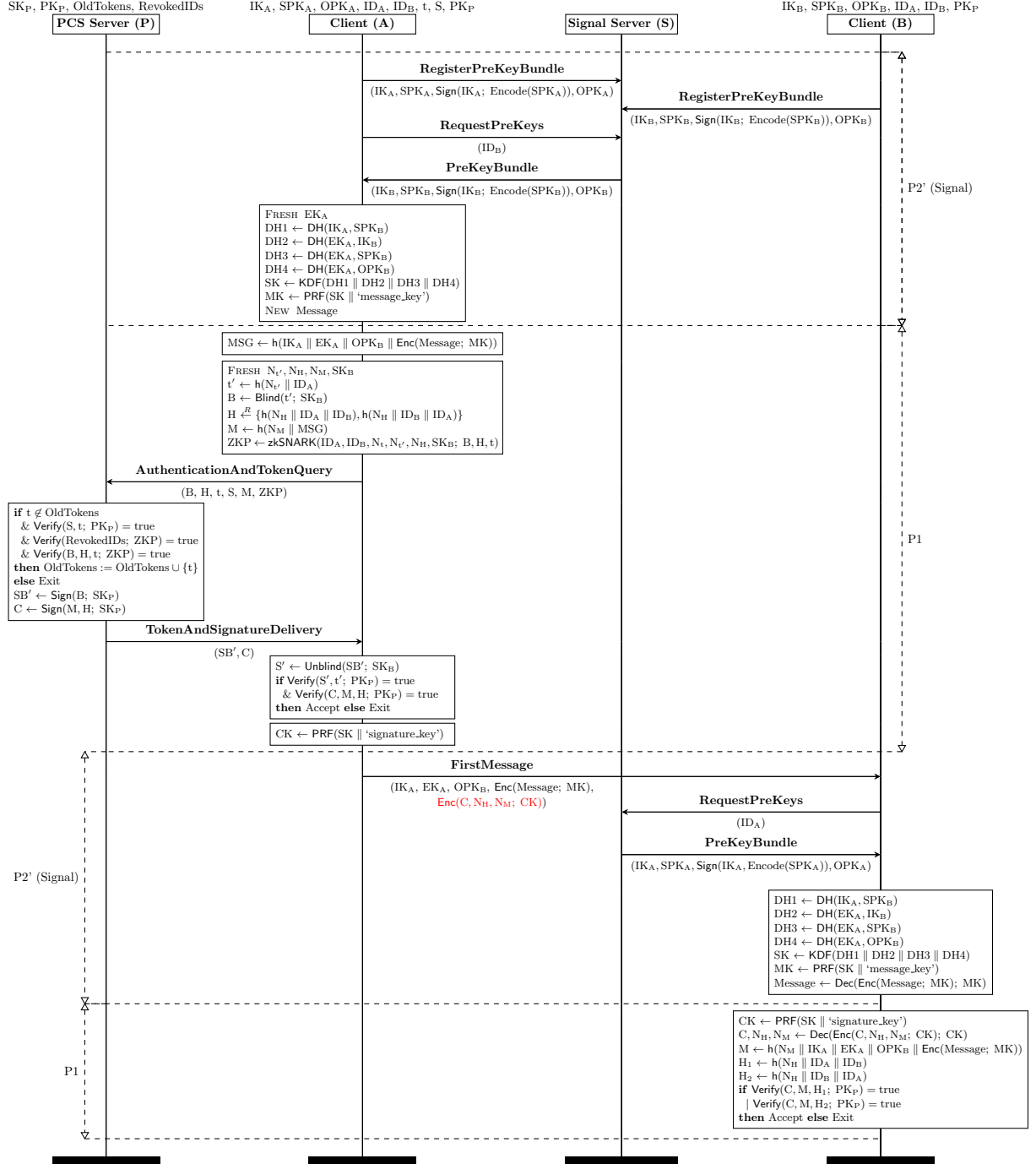


Figure 3: 3PaaS with X3DH [50] as the P2 protocol. Red text in the P2' part indicates changes to the original protocol. Specifically, we add two nonces and an encrypted signature to the first message sent from A to B.

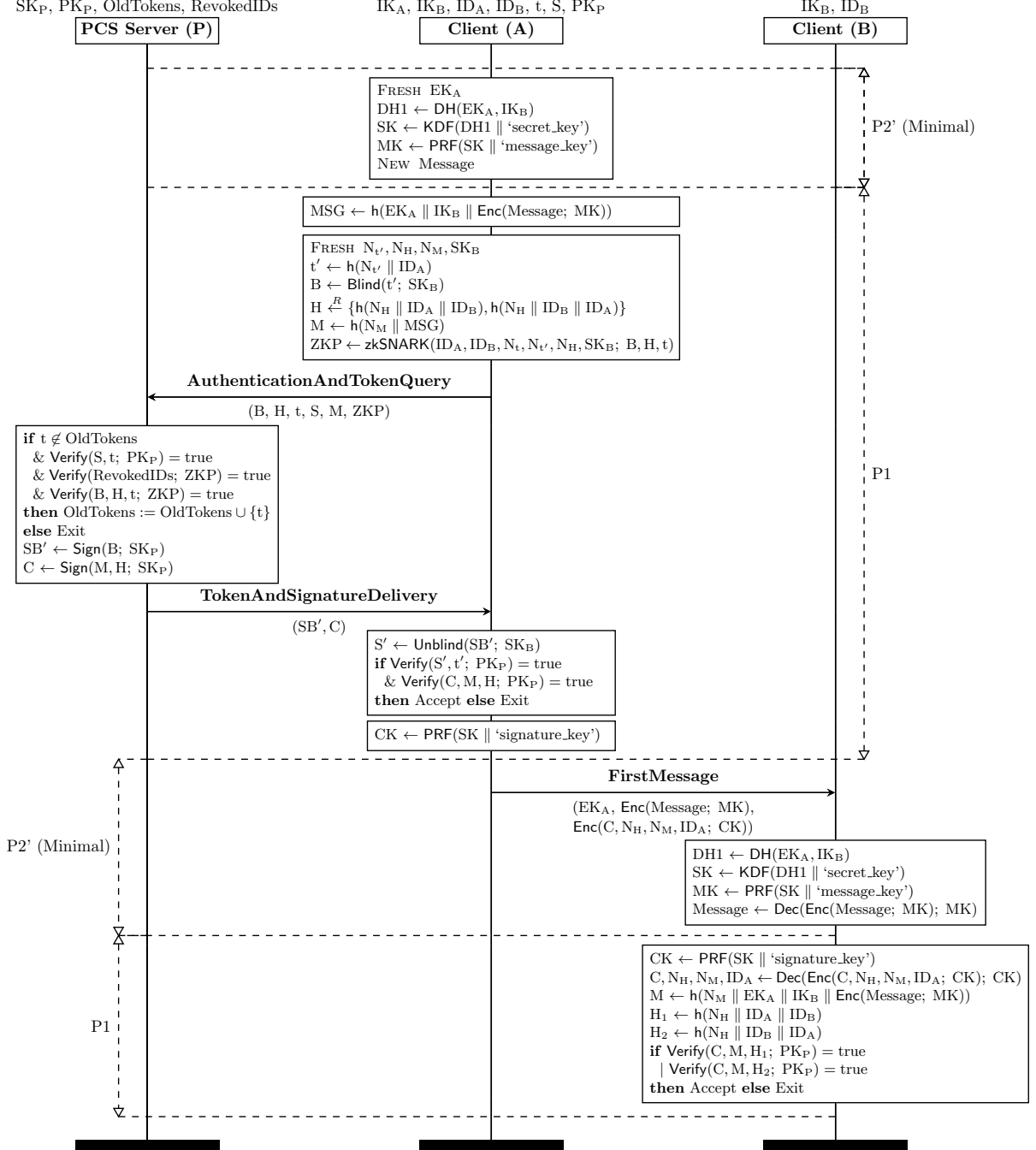


Figure 4: 3PaaS with a simplified version of X3DH as the P2 protocol.

---

**Algorithm 4** The full Blind functionality.

---

```

function MGF( $S, emLen - hLen - 1$ ):
  if ( $emLen - hLen - 1$ ) >  $2^{32}$  then
    return  $\perp$ 
   $T \leftarrow \{\}$ 
  for  $i = 0$  to  $\lceil (emLen - hLen - 1/hLen) - 1 \rceil$  do
    // integer to octet representation
     $C \leftarrow \text{I2OSP}(i, 4)$ 
     $T = T \parallel h(S \parallel C)$ 
  return leading ( $emLen - hLen - 1$ ) bytes of  $T$ 

function Encode( $mHash, salt$ ):
   $emLen \leftarrow 256$  // length of  $em$  in bytes
   $sLen \leftarrow 15$  // length of salt in bytes
   $hlen \leftarrow 32$  // length of hash digest in bytes
   $emBits \leftarrow 2047$  // 1 less than RSA modulus bits
   $M' \leftarrow \{(0x) 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\} \parallel mHash \parallel salt$ 
   $S \leftarrow h(M')$ 
  // padding string of zeros
   $PS \leftarrow 0x00 \cdot (emLen - sLen - hlen - 2)$ 
   $DB \leftarrow \{PS \parallel 0x01 \parallel salt\}$ 
   $dbMask \leftarrow \text{MGF}(S, emLen - hLen - 1)$ 
   $maskedDB \leftarrow \{DB \oplus dbMask\}$ 
  // set the leftmost ( $8 \cdot emLen - emBits$ ) bits of the leftmost octet in maskedDB to zero
   $em \leftarrow \{maskedDB \parallel S \parallel 0xBC\}$ 
  return  $em$ 

function Blind( $mHash, (e, N), salt$ ):
   $em \leftarrow \text{Encode}(mHash, salt)$ 
   $r \xleftarrow{R} Z_N^*$ 
   $SK_B \leftarrow r^e \bmod N$ 
   $B \leftarrow em \cdot SK_B \bmod N$ 
  return  $B, r$ 

function UnBlind( $BlindSig, r, N$ ):
   $Sig \leftarrow BlindSig \cdot r^{-1} \bmod N$ 
  return  $Sig$ 

```

---

## D RSA Blind Signature Encode Algorithm

Algorithm 4 describes the full details of the RSA blind signature algorithm we use (as defined in RFC 9474 [31]), including the encoding function and the hash-based mask generation function MFG defined in [53]. Algorithm 5 is a pseudocode of the ZK circuit we used (and described in Section 5.4), which includes the implementation of the encoding function.

---

**Algorithm 5** ZKP circuit for the unlinkable version of 3PaaS. The function takes as input a public and a private witness, as described in Section 5.4.

---

```

function Encode( $mHash$ ,  $salt$ ):
     $emLen \leftarrow 256$  // length of  $em$  in bytes
     $sLen \leftarrow 15$  // length of salt in bytes
     $hlen \leftarrow 32$  // length of hash digest in bytes
     $emBits \leftarrow 2047$  // 1 less than RSA modulus bits
     $M' \leftarrow \{(0x) 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00\ 00 \parallel mHash \parallel salt\}$ 
     $S \leftarrow h(M')$ 
     $PS \leftarrow \{0x00\} \times 207$  // 207 octets of 0x00
     $DB \leftarrow \{PS \parallel 0x01 \parallel salt\}$ 
     $dbMask \leftarrow \text{MGF}(S, 223)$ 
     $maskedDB \leftarrow \{DB[: -31] \parallel (DB[-31:] \oplus dbMask[-31:])\}$ 
    // set the leftmost  $(8 \cdot emLen - emBits)$  bits of the leftmost octet in maskedDB to zero
     $em \leftarrow \{maskedDB \parallel S \parallel 0xBC\}$ 
    return  $em$ 

function 3PaaS( $(t, B, H, RT)$ ,  $(r, N_t, N_{t'}, ID_A, ID_B, N_H, b, salt)$ ):
     $\tilde{t} \leftarrow h(N_t \parallel ID_A)$ 
    // verify that  $t \leftarrow h(N_t \parallel ID_A)$ 
    assert ( $t = \tilde{t}$ )
     $t' \leftarrow h(N_{t'} \parallel ID_A)$ 
     $em \leftarrow \text{Encode}(t', salt)$ 
     $\tilde{B} \leftarrow (em \cdot r^e) \bmod N$ 
    // verify that  $B \leftarrow \text{Blind}(h(N_{t'} \parallel ID_A); SK_B)$ 
    assert ( $B = \tilde{B}$ )
    if  $b = 1$  then
         $\tilde{H} = h(N_H \parallel ID_A \parallel ID_B)$ 
    else if  $b = 0$  then
         $\tilde{H} = h(N_H \parallel ID_B \parallel ID_A)$ 
    // verify that  $H \in \{h(N_H \parallel ID_A \parallel ID_B), h(N_H \parallel ID_B \parallel ID_A)\}$ 
    assert ( $H = \tilde{H}$ )
    // verify that  $ID_A$  is not revoked (see Section 5.3 for details)
    assert ( $\text{NotRevoked}(ID_A, RT)$ )
    return 1

```

---